

M7 · the paradigm shift

Day 1

Paradigm, not feature tour

35 min

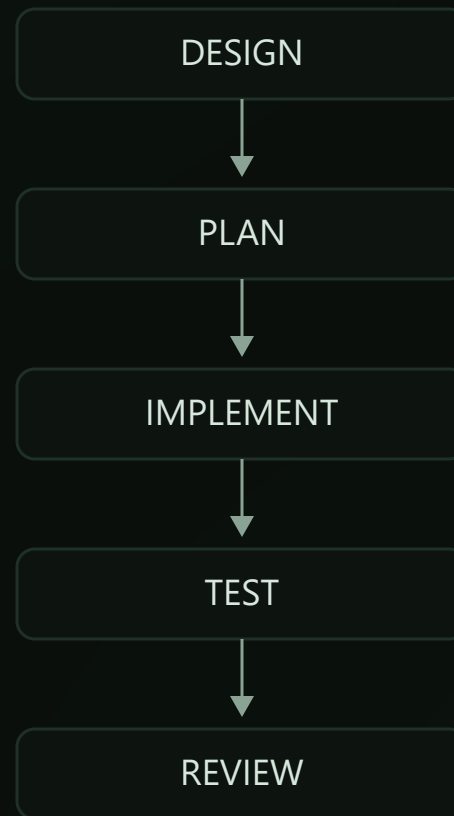
From Assistant to Agent

The paradigm shift

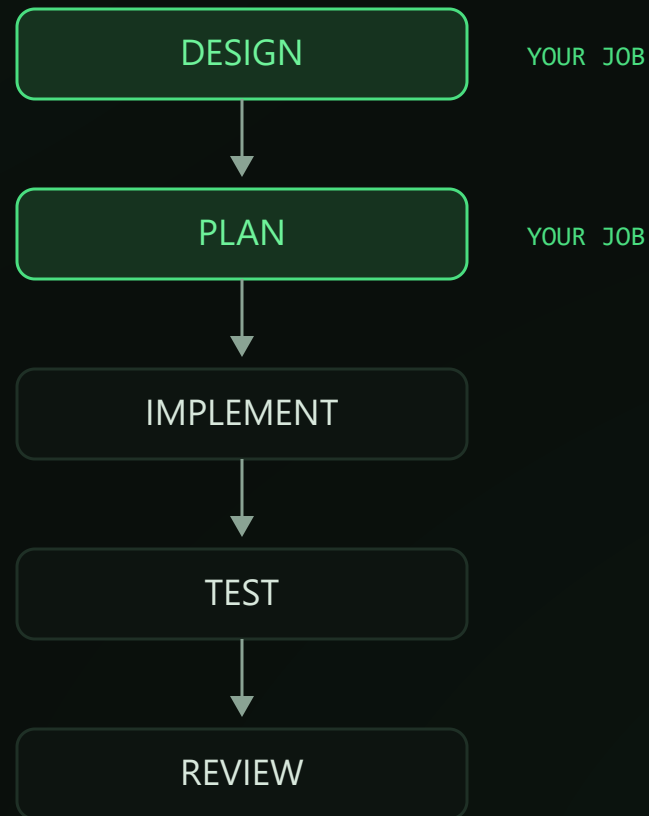
Thesis: An agent is not a smarter autocomplete. It is the loop you already run — **Design** → **Plan** → **Implement** → **Test** → **Review** — with the bottom of the loop automated.

Bridge from M5: context fed once becomes context that can feed itself.

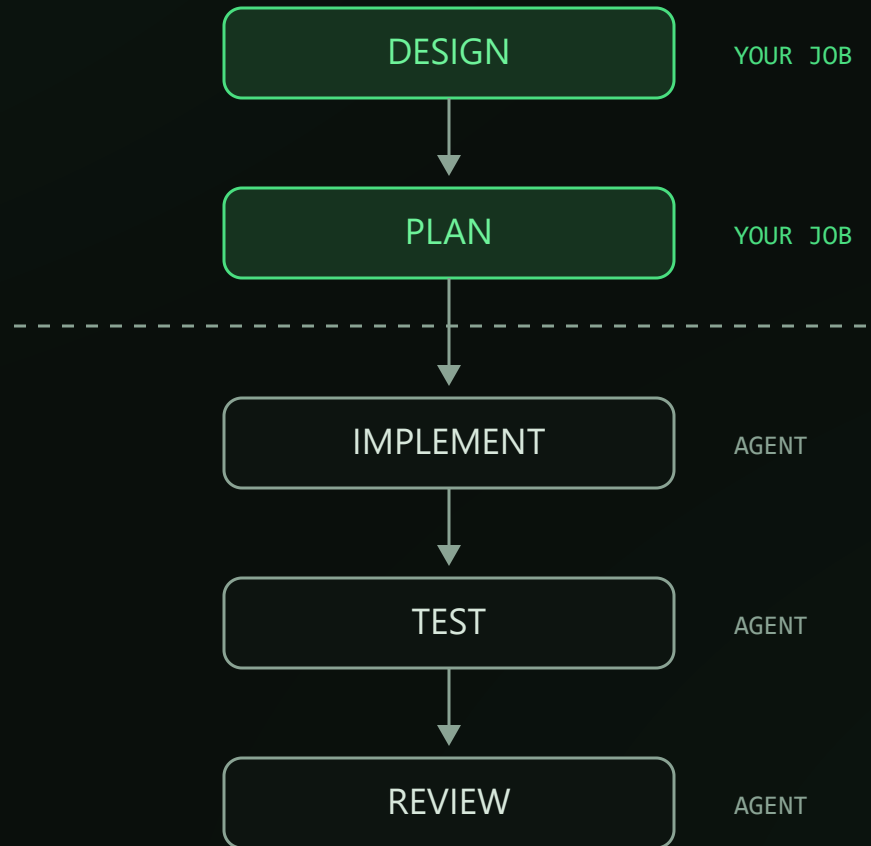
The loop you already run



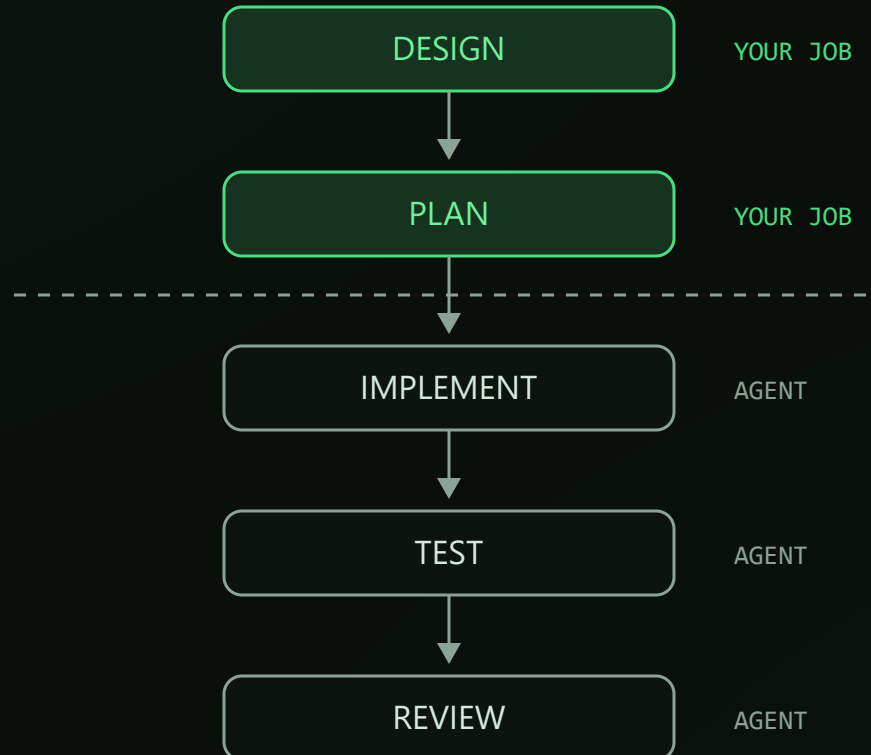
The top of the loop stays human



Below the line, the speed changes



The split



'The things in green are what's left for you.' — Adam Jacob

What changes, what stays



What dies

- Reading every line of code
- Manual PR review at scale
- Style debates, naming bikesheds, DRY theater

What survives

- **Architecture** — keeps the agent coherent
- **Taste** — tells you what good looks like
- **Domain expertise** — the agent knows code; you know your infrastructure

Spot-check question

Did the output follow the design, the plan, and the standards — or did it drift?

Show, don't tell

Recorded demo clip

Task: add input validation to a Terraform module, run validation, fix the failure, and pass.

```
> Add input validation to this Terraform module. # 1. plan
mode - inspect variables.tf and README - add validation
blocks for environment and name length - update examples and
docs - run terraform validate # 2. multi-file edit
variables.tf README.md examples\basic\main.tf # 3. test +
self-correct terraform validate Error: invalid default value
agent patches variable default terraform validate Success!
The configuration is valid.
```

What to watch for

The plan appears before the code. That is your review point.

Then the bottom of the loop runs

Implement → test → interpret failure → fix → test again.

What you did not do

You did not type every line. You set direction, approved the path, and let the loop run.

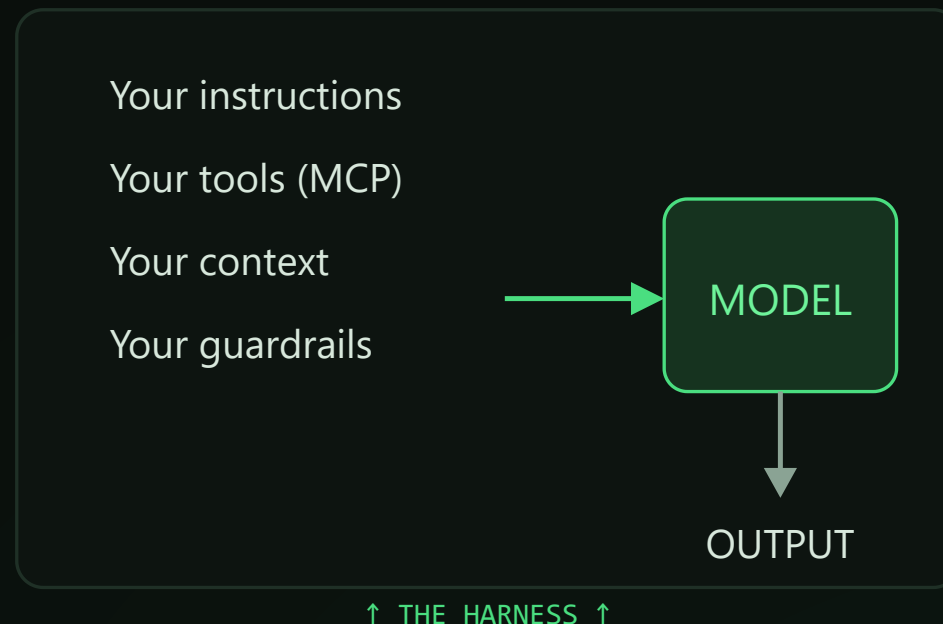
The harness is the leverage

Harness > model

Instructions, tools, context, memory, and guardrails determine whether the model is useful.

Architecture keeps the agent coherent. The harness makes the architecture reachable.

— Synthesis from Adam Jacob + Peter Steinberger



AI-native automation · 1 of 5

Transparency

The agent must be able to introspect every step.

Why it matters

If you hide the state, the model fills gaps with guesses.

Terraform example

`terraform plan` is useful because the upcoming change is visible and machine-readable.

AI-native automation · 2 of 5

Extensibility

The agent should be able to write the missing piece instead of waiting on a roadmap.

Why it matters

Work stops being blocked by 'that integration doesn't exist yet.'

Proof point

In Adam Jacob's demo, the agent learned Proxmox by building its own extension model.

AI-native automation · 3 of 5

One-to-one to the domain

The closer your tooling maps to the real domain, the easier it is for the agent to stay correct.

Why it matters

The model was trained on the world as it is, not on your private wrapper language.

Implication

Prefer thin abstractions and explicit Terraform resource intent over internal aliases/abstractions.

AI-native automation · 4 of 5

Validation everywhere

When the work can spend money or hurt people, policy must travel with the loop.

Terraform example

Validation blocks, policy checks, and pre-commit hooks become tools the agent runs as part of the loop.

What changes

The agent does not replace guardrails. It makes them more important.

AI-native automation · 5 of 5

Full lifecycle visibility

The loop cannot stop at deploy. The agent needs visibility from design through operations.

Why it matters

Disconnected tools create blind spots the agent has to guess across.

Implication

Consolidate the source of truth so runbooks, IaC, telemetry, and troubleshooting are part of the same story.

The taste warning

“ If you fully specified it, you would have used keywords in a programming language — and you would control the thing yourself. ”

Mark Russinovich · public *Scott & Mark Learn To ... Have Taste* appearance

Warning: Agents amplify both good and bad judgment. If your architecture is sloppy, the slop just arrives faster.

⚠ Do not use internal company data outside approved governance.

Proof of concept · Lobster Curl 0.1



The setup

Peter Steinberger dropped his agent into a tiny Arch Linux container with no curl, no wget, and basically nothing useful inside.

What the agent did

It found a compiler, opened a raw socket, and wrote its own HTTP client in C.

What landed

Lobster Curl 0.1 fetched web pages from a container that started with no web client at all.

The punchline

Design: fetch web data. Plan: need an HTTP client. Implement: write one. Test: does it fetch? Review: it works. **That's not autocomplete. That's an agent.**

M7 · the paradigm shift

Day 2 starts here

M10 · Agent Mode Deep Dive

You will configure agents, wire up tools, and run the bottom of the loop yourselves.

Bring one real workflow

Find one place in your daily work where you still do **Design → Plan → Implement → Test → Review** manually.

Terraform module · PowerShell migration · documentation update · anything you actually care about

Come back hungry.